Prof. Ignacio Traverso y Sofía Mármol

Algoritmos y Estructuras de Datos

Archivos de texto



Contenidos del documento:

- Lectura de texto
- Escritura de texto
- Validacion
- Que es y para que sirve el tipo de archivo JSON
- Lectura de archivos JSON
- Escritura de archivos JSON

Lectura y Escritura de texto:

El espacio de nombres ***System.IO*** de C# provee las clases ***StreamReader*** y ***StreamWriter*** para la lectura y escritura secuencial de flujos de texto.

StreamReader permite acceder a un archivo de texto carácter por carácter o línea por línea desde el principio hasta el final. Se recomienda envolver el objeto en un bloque `using` para asegurar la liberación del archivo en el sistema operativo una vez concluida la lectura.

¿ qué es using?

Asegura que los recursos externos (como archivos abiertos, conexiones a bases de datos o memoria del sistema) se **cierren y liberen automáticamente** al terminar de usarlos, evitando archivos bloqueados o fugas de memoria

```
using System;
using System.IO;

string rutaEntrada = "archivo.txt";

using (StreamReader reader = new StreamReader(rutaEntrada))
{
    // Lee el contenido completo del archivo de una sola vez
    string contenidoCompleto = reader.ReadToEnd();
    Console.WriteLine(contenidoCompleto);
}
```

Escritura:

`StreamWriter` se utiliza para escribir caracteres en un flujo de texto. Su constructor acepta un parámetro booleano (`append`) que determina si el contenido se añadirá al final del archivo (`true`) o si el archivo se sobrescribirá desde cero (`false`).

```
using System;

using System.IO;



string rutaSalida = "salida.txt";



// append: false sobrescribe el archivo; append: true agrega al final

using (StreamWriter writer = new StreamWriter(rutaSalida, append: false))

{

    writer.WriteLine("Encabezado del archivo");

    writer.WriteLine("Línea 1: Registro guardado correctamente.");

    writer.Write("Línea 2: Texto sin salto de línea... ");

    writer.WriteLine("continuación de línea 2.");

}
```

Validacion de fin de archivo:

Para procesar archivos línea por línea secuencialmente sin intentar leer más allá del límite del archivo, existen dos mecanismos principales para validar el fin de archivo.

Opción A: Propiedad EndOfStream

Devuelve true cuando se ha alcanzado la posición final del flujo.

```
using (StreamReader reader = new StreamReader("datos.txt"))
{
    while (!reader.EndOfStream)
    {
        string? linea = reader.ReadLine();
        Console.WriteLine(linea);
    }
}
```

Opción B: Evaluación del retorno null en ReadLine()

El método ReadLine() retorna null cuando no hay más caracteres para leer.

```
using (StreamReader reader = new StreamReader("datos.txt"))
{
    string? linea;
    while ((linea = reader.ReadLine()) != null)
    {
        Console.WriteLine(linea);
    }
}
```

Lectura y Escritura de Archivos JSON:

Un archivo JSON (*JavaScript Object Notation*) es un formato de texto ligero y estandarizado que se utiliza para almacenar y transportar datos de manera legible tanto para personas como para computadoras.

¿Para qué se utiliza?

- Comunicación con APIs: Permite que una página web o aplicación móvil hable con un servidor y envíe o reciba información al instante.
- Archivos de configuración: Muchas aplicaciones guardan ahí las preferencias del usuario, como el idioma o el modo oscuro.
- Almacenamiento: Se usa en bases de datos NoSQL como MongoDB

Para trabajar con **archivos de texto en formato JSON** en C#, la opción estándar y recomendada es el espacio de nombres nativo **`System.Text.Json`**.

Para estructurar la información que guardaremos en el archivo JSON, creamos una clase o estructura con las propiedades requeridas:

```
public class Producto

{

    public int Id { get; set; }

    public string Nombre { get; set; } = string.Empty;

    public double Precio { get; set; }

    public bool EnStock { get; set; }

}
```

El proceso de convertir un objeto de C# a texto JSON se denomina Serialización.

Método A: Usando `File.WriteAllText` (Forma directa)

```
using System;

using System.IO;

using System.Text.Json;



class Program

{

    static void Main()

    {

        // 1. Crear el objeto o lista de datos

        var producto = new Producto

        {

            Id = 1,

            Nombre = "Laptop Gamer",

            Precio = 1250.99,

            EnStock = true

        };
```

```
 // 2. Configurar opciones de formato (ej. sangría / formato legible)

  var opciones = new JsonSerializerOptions

        {

            WriteIndented = true // "Pretty printing" para que se lea con
saltos de línea

        };



        // 3. Serializar el objeto a cadena JSON

        string jsonTexto = JsonSerializer.Serialize(producto, opciones);



        // 4. Guardar la cadena en el archivo

        string rutaArchivo = "producto.json";

        File.WriteAllText(rutaArchivo, jsonTexto);



        Console.WriteLine($"Archivo JSON creado correctamente en:
{rutaArchivo}");

    }

}
```

Resultado:

El proceso de convertir un objeto de C# a texto JSON se denomina Serialización.

```
{
  "Id": 1,
  "Nombre": "Laptop Gamer",
  "Precio": 1250.99,
  "EnStock": true
}
```

Método B: Usando `StreamWriter` o Flujos Asíncronos (`FileStream`)

Para archivos de mayor tamaño o dentro de arquitecturas asíncronas (`async/await`), es más eficiente escribir directamente sobre el *stream*:

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Text.Json;
using System.Threading.Tasks;


class Program
{
    static async Task Main()
    {
        var listaProductos = new List<Producto>
        {
```

```
            new Producto { Id = 1, Nombre = "Monitor 27''", Precio =
299.99, EnStock = true },
            new Producto { Id = 2, Nombre = "Teclado Mecánico", Precio =
85.50, EnStock = true },
            new Producto { Id = 3, Nombre = "Mouse Inalámbrico", Precio =
35.00, EnStock = false }
        };

        string ruta = "inventario.json";
        var opciones = new JsonSerializerOptions { WriteIndented = true };

        // Abrir/crear el archivo y escribir asíncronamente en el stream
        using (FileStream stream = File.Create(ruta))
        {
            await JsonSerializer.SerializeAsync(stream, listaProductos,
opciones);
        }

        Console.WriteLine("Lista guardada en inventario.json
exitosamente.");
    }
}
```

Para leer un archivo JSON existente y convertir su contenido de vuelta a objetos de C#, se utiliza **Deserialización**.

```
using System;

using System.Collections.Generic;

using System.IO;

using System.Text.Json;



class Program

{

    static void Main()

    {

        string ruta = "inventario.json";



        if (File.Exists(ruta))

        {

            // 1. Leer el texto completo del archivo

            string contenidoJson = File.ReadAllText(ruta);



            // 2. Opciones de deserialización (ej. ignorar
mayúsculas/minúsculas)

            var opciones = new JsonSerializerOptions

            {

                PropertyNameCaseInsensitive = true

            };
```

```
            // 3. Convertir de JSON a objeto de C#
            List<Producto>? productos =
JsonSerializer.Deserialize<List<Producto>>(contenidoJson, opciones);


            if (productos != null)
            {
                foreach (var p in productos)
                {
                    Console.WriteLine($"[ID: {p.Id}] {p.Nombre} -
${p.Precio:F2} (Stock: {p.EnStock})");
                }
            }
        }
    }
}
```